

Getting started

This class will emphasize UNIX skills to support doing genomics and evolutionary analysis with bioinformatics tools. There are many many tutorials and workshops out there. Many are available for free and linked here

- [Data Carpentry](#) and [Software Carpentry](#) part of [The Carpentries](#)
- [Data Intensive Biology training](#) like [Shell Genomics](#)
- [Getting Started with Genomics Tools](#)
- [Happy Belly Bioinformatics](#)

Logging into Cluster

You will need to have a terminal to get onto the system. On Mac that is called 'Terminal'. On Windows [MobaXTerm](#) is the best tool. Choose the 'Free' and 'Portable Version'.

Existing documentation and tutorial available at <http://hpcc.ucr.edu/> for using the HPCC.

See the [Linux Basics](#)

To login to the cluster we need to use ssh client. This allows secure communication with the cluster. The UCR cluster is accessed using the host `cluster.hpcc.ucr.edu`.

```
$ ssh -X USERNAME@cluster.hpcc.ucr.edu
```

This will initiate a [UNIX](#) session running on the cluster 'head node' by connecting through a secure connection. There are multiple machines which serve as this login node where we can stage our analysis to run on the worker nodes that are on the cluster so you may see different names like 'pelican', 'pigeon' when you log in each time. Much more detail on the setup of the cluster and resources available at <http://hpcc.ucr.edu>.

You should now see a message as well as a prompt:

```
-----  
University of California, Riverside - HPCC (High-Performance Computing Center)  
-----
```

More information about HPCC and how to use the resources provided can be found at http://hpcc.ucr.edu/manuals_linux-cluster_intro.html

Please send all questions and support requests to support@hpcc.ucr.edu

Note: The default version of R is now 3.6.0

```
-----  
username@pelican:~$
```

Let's setup some initial things. Make a SSH folder so we can copy thing over.

```
[hpcc] $ mkdir ~/.ssh  
[hpcc] $ chmod 700 ~/.ssh # sets the permission for this folder
```

You want to change your password on the cluster to something only you know. This is good practice since your password was emailed to you. It will prompt you for your current password and a new one.

```
[hpcc] $ passwd
```

You can log off by typing

- `exit`
- `␣` (Control-D)

The prompt on our system by default will start with the name of the computer as well as the current directory you are logged into. This prompt like most things on the system can be customized.

```
hostname:[directory]$
```

The `-X` option tells the system to [forward your X11 connection](#) which is necessary for running interactive graphics (eg showing an image, running a graphical editor program like emacs)

The command line interface (CLI)

The command line provides ability to interact with the filesystem (files and folders) and run programs. A collection of UNIX utilities.

Directories and files

ls - list the files and folders in a directory. Options include `-l` to list with details (long). `-t` list ordered by time created (time). These can be combined as `ls -lt` or `ls -l -t`. Specify a folder to list other than the current directory with another argument `ls -l data`.

mkdir Create a directory. Give the `-p` option to create any necessary sub folders and also to not give warnings if a folder already exists.

```
$ mkdir test  
$ mkdir Alpha/Beta/Zeta # will give error  
$ mkdir -p Alpha/Beta/Zeta # will not give error
```

rmdir Remove a folder. Only works if folder is empty

rm Remove a file or folder. This is command to be careful with. To delete a folder that contains many folders you can use.

```
rm -rf FOLDER # USE WITH CAUTION, THIS WILL RECURSIVELY DELETE
```

cd To navigate into a directory. Use this to go to different directories.

- **cd dirname** - go to a directory
- **cd /bigdata/gen220/shared** - can be an absolute path
- **cd simple** - or a relative directory (eg the dir **simple** that is in the folder).
- **cd ..** - go up a directory
- **cd** or **cd ~** - go to the home directory
- **cd -** - go to the last directory

pushd/popd - like **cd** but keep track of all the directories you were in.

```
[~] $ pushd /bigdata/gen220
[/bigdata/gen220] $ # now you are in a new directory
[/bigdata/gen220] $ pushd /tmp
[/tmp] $ # now you are in another directory
[/tmp] $ cd # go back to your home dir
[~] $ popd
[/bigdata/gen220] $ # popd removes the top of the stack and returns you to the next directory
[/bigdata/gen220] $ popd
[~] $ # back to the directory you started in
```

This is different from **cd -** because it remembers the last directory you were in when you issued **pushd**. So you can use it like a bookmark to get you back to a specific place despite any of the other commands.

pwd Print the current working directory. Helpful to remember where you are.

pwd

realpath Print out the full path to a file. If it is a symlink print out where the original file is located.

```
$ cd /bigdata/gen220/shared/simple
$ ls -l
lrwxrwxrwx 1 jstajich gen220    20 Sep 30 12:06 gene_names.txt -> yeast_gene_names.txt
-rw-r--r-- 1 jstajich gen220   603 Oct 10  2018 numbers_floating.dat
-rw-r--r-- 1 jstajich gen220 22447 Oct 10  2018 rice_random_exons.bed
-rw-rw-r-- 1 jstajich gen220 33894 Sep 30 12:05 yeast_gene_names.txt
$ realpath yeast_gene_names.txt
/bigdata/gen220/shared/simple/yeast_gene_names.txt
$ realpath gene_names.txt
/bigdata/gen220/shared/simple/yeast_gene_names.txt
```

more See the contents of a text file, one page at a time. Go to the next page with 'space'. Can search for a specific text with slash (/).

```
$ more /bigdata/gen220/shared/simple/yeast_gene_names.txt
```

less See the contents of a text file, one page at a time. Less has *more* options than **more** with arrows which will let you navigate up and down pages and a

search option - use the slash (/) and then type in a search text it will highlight all the options.

```
$ less /bigdata/gen220/shared/simple/yeast_gene_names.txt
```

head see the first lines in a file. By default this is 10 lines. But you can specify as many as you want with **-n LINES** option. Useful to get the beginning of a report or see what is the header in a spreadsheet file.

```
# here's an example that will work on the cluster
```

```
head -n 15 /bigdata/gen220/shared/simple/numbers_floating.dat
```

tail see the last lines in a file. By running this command can see the last 10 lines by default. Can specify number of lines with **-n LINES** option. Useful when looking at a log-file and want to see the last reported messages.

```
tail -n 12 FILE.txt
```

```
# here's an example that will work on the cluster
```

```
tail -n 3 /bigdata/gen220/shared/simple/numbers_floating.dat
```

echo Prints out messages.

```
echo "hello there"
```

```
# if you want to use special characters like tab (\t) you need to specify -e when you run echo
```

```
echo -e "Chrom\tStart\tEnd"
```

Writing and running a shell script

A **shell script** is a text file containing the commands you would type at the prompt. Putting your commands in a script means you can re-run the whole analysis with one command, fix a mistake and run it again, and share exactly what you did. All the homework in this class will be turned in as scripts.

Make a small data file to practice on

```
mkdir -p ~/bigdata/gen220/scripts_practice
```

```
cd ~/bigdata/gen220/scripts_practice
```

```
echo -e "YAL001C\tTFC3" > genes.txt # > creates (or overwrites) the file
```

```
echo -e "YAL002W\tVPS8" >> genes.txt # >> adds to the end of the file
```

```
echo -e "YAL003W\tEFB1" >> genes.txt
```

```
cat genes.txt
```

```
YAL001C TFC3
```

```
YAL002W VPS8
```

```
YAL003W EFB1
```

Write the script

Open a new file in a text editor. **nano** is the simplest editor on the command line (or use the Jupyter or VS Code editors described below):

```
nano count_lines.sh
```

Type in these lines. Save with **Ctrl-O** (then Enter) and exit with **Ctrl-X**.

```
#!/usr/bin/env bash
# count_lines.sh - report how many lines are in a file
echo "Counting the lines in genes.txt"
wc -l genes.txt
```

- The first line, starting with **#!** (called the “shebang”), tells the computer which program should run this file - here, **bash**.
- Every other line starting with **#** is a **comment**. It is ignored when the script runs; it’s a note for people reading it (including you in a month).
- By convention, shell scripts end in **.sh**.

Run the script

There are two ways. The first is to pass the file to **bash**:

```
bash count_lines.sh

Counting the lines in genes.txt
3 genes.txt
```

The second way is to make the script **executable** (give it the **x** permission) with **chmod**, then run it directly:

```
chmod +x count_lines.sh
ls -l count_lines.sh      # the x's in -rwxr-xr-x mean it can be executed
./count_lines.sh
```

Why **./**? When you type a command name like **ls**, the shell looks for a program with that name in a list of folders called your **PATH** (see it with **echo \$PATH**). Your current folder is not on that list, so you have to say where the script is: **./** means “in this folder”. Without it you get **command not found**.

Variables and arguments

A script that only works on **genes.txt** isn’t very useful. Scripts can use **variables** and take **arguments** from the command line, so the same script works on any file:

```
#!/usr/bin/env bash
# file_summary.sh - print the number of lines and the first lines of a file
# usage: ./file_summary.sh FILENAME

if [ $# -lt 1 ]; then
    echo "usage: $0 FILENAME"
    exit 1
fi
```

```

FILE=$1
LINES=$(wc -l < "$FILE")
echo "$FILE has $LINES lines"
echo "The first 2 lines are:"
head -n 2 "$FILE"

chmod +x file_summary.sh
./file_summary.sh genes.txt

genes.txt has 3 lines
The first 2 lines are:
YAL001C TFC3
YAL002W VPS8

```

- `FILE=$1` creates a variable called `FILE`. There must be **no spaces** around the `=`: `FILE = $1` is an error, because bash thinks you are running a command called `FILE`.
- `$1` is the first argument given after the script name, `$2` the second, and so on. `$#` is the number of arguments, and `$0` is the name of the script itself.
- Use `$FILE` (with a `$`) to get the value back out. Put it in double quotes, `"$FILE"`, so file names containing spaces still work.
- `$(command)` runs a command and captures its output - here, the line count from `wc -l`.
- The `if [ ... ]; then ... fi` block checks that an argument was given, and prints a usage message and stops (`exit 1`) if not. We will cover `if` statements and loops in more detail in later UNIX lectures.

Common problems

- **Permission denied** when running `./myscript.sh` - you forgot `chmod +x myscript.sh` (or use `bash myscript.sh`).
- **command not found** - you left off the `./`, or you have spaces around an `=`.
- **bad interpreter** or an error mentioning `\r` or `^M` - the file was saved with Windows line endings (e.g. edited in Notepad on Windows). Fix it on the cluster with `dos2unix myscript.sh`, and set your editor to use UNIX (LF) line endings.
- The script ran but did the wrong thing - run it with `bash -x myscript.sh` to print each command as it runs.

Practice: write a script called `count_genes.sh` which takes a file name as an argument and prints how many lines in the file contain the text `YAL` (hint: `grep -c`). Run it on `genes.txt`.

Logging in with SSH keys

See the [SSH keys guide](#) for a quick start, diagrams of which files go where, Windows instructions, and troubleshooting.

Let's make it simpler to login to the cluster without having to type our password all the time. This is a **ONE TIME CONFIGURATION**, you don't need to do this every time you connect, you only will need to setup for your laptop to connect using these keys. These keys are also useful when we start using github to commit your code as it will also use this connection mechanism. If you are using an in-class laptop you'll need to keep using that same one over the course of the class or re-do this step.

If you are using mobaxterm or other system that will SAVE your password then you really don't have to do this step for ssh keys for connecting (though it will still be useful for the github connecting).

SSH-keys will allow you to setup connection to the server without having to use your server password each time. You can specify a key-pair that will work for your laptop to connect to the server.

Setup your SSH access on your computer (your laptop).

Generate RSA key pair on your computer. It will ask you for a password. You get to pick any password here you can remember. This will be a different password from your cluster one.

```
[your laptop] $ ssh-keygen -t rsa
```

See the new files created

```
[your laptop] $ ls ~/.ssh/  
id_rsa id_rsa.pub
```

Let's also configure how our ssh works by customizing the ssh config. We need to edit a text file. On OSX this can be with `vi`, `emacs`, `nano`

```
nano ~/.ssh/config
```

```
ForwardX11 yes  
ForwardX11Trusted yes  
ForwardAgent yes
```

```
Host hpcc  
  Hostname cluster.hpcc.ucr.edu  
  User YOURHPCCUSERNAME  
  ServerAliveInterval 10
```

This will open up a window for editing. You should add this content to your file. Here's [a file you can copy onto the server too](#).

```
scp ssh-config_example.txt YOURHPCCUSERNAME@cluster.hpcc.ucr.edu:~/.ssh
```

Now you need to copy your ssh key FROM your laptop TO the cluster.

```
[your laptop] $ scp ~/.ssh/id_rsa.pub YOURHPCCUSERNAME@cluster.hpcc.ucr.edu:~/.ssh/my_laptop_id_rsa.pub  
[your laptop] $ ssh hpcc
```

```
[hpcc] $ cat ~/.ssh/my_laptop_key.pub >> ~/.ssh/authorized_keys
[hpcc] $ chmod 600 ~/.ssh/authorized_keys
```

Web Access with Jupyter

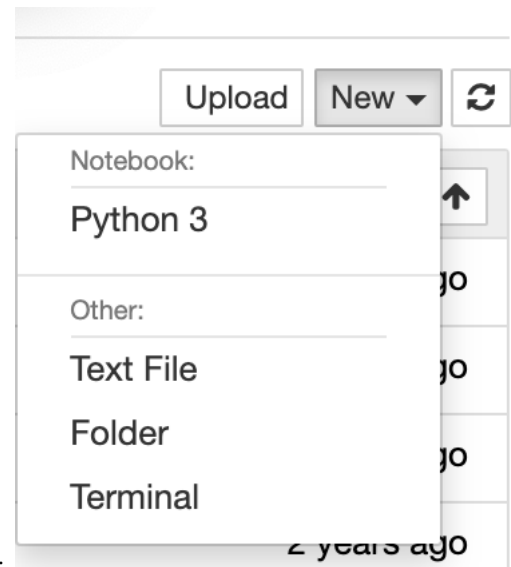
[Jupyter notebooks](#) are ways to run Python or programming languages within a web environment. There are also utilities for command line access.

You can access the HPCC instance at <https://jupyter.hpcc.ucr.edu/>.

This can also be easily installed and run on your own computer or laptop if you prefer the web interface for your coding. There are also several [free tools](#). The challenges in these is to be able to save the code you write in these virtual environments for later. You can connect to

Finally I don't advocate using the jupyter notebooks for more than a testing or learning environment. The code you will turn in for class homework will need to be simple python or shell code (eg text files) not ipython notebook binary files.

Jupyter notebook after logging in:

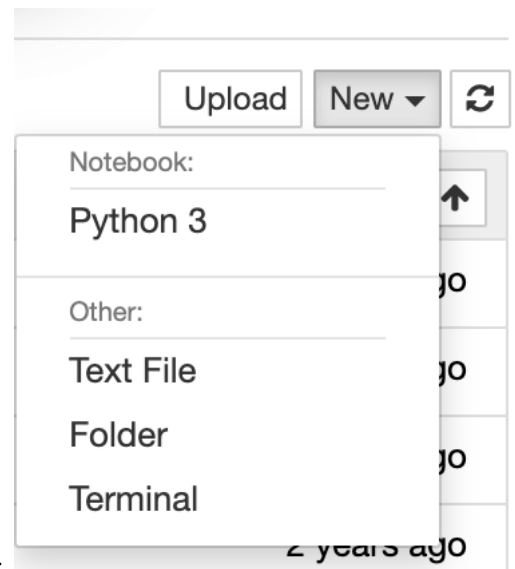


Start a Terminal Session - select the *Terminal* option:

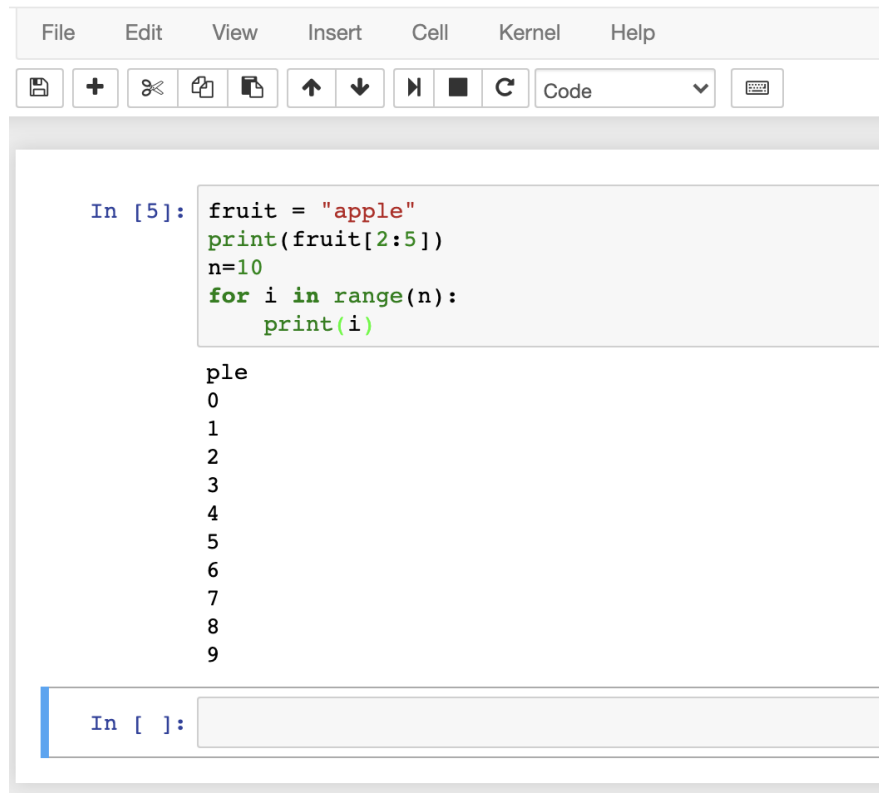
Running in the terminal (on HPCC) so you can see files on your account there:



```
bash-4.2$ pwd
/rhome/jstajich
bash-4.2$ cd bigdata/
bash-4.2$ pwd
/rhome/jstajich/bigdata
bash-4.2$
```



Start a Python Notebook Session - select the *Python 3* option:



The screenshot shows a Jupyter Notebook interface. At the top is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', and 'Help'. Below the menu is a toolbar with icons for saving, adding, deleting, copying, pasting, navigating, and running code. The main area contains a code cell with the following Python code:

```
In [5]: fruit = "apple"
        print(fruit[2:5])
        n=10
        for i in range(n):
            print(i)
```

The output of the code is displayed below the code cell:

```
ple
0
1
2
3
4
5
6
7
8
9
```

At the bottom, there is an input prompt 'In []:' followed by a text box for entering new code.

Enter some Python code and run it:

Rstudio

For some part of the class we will use Rstudio which is a web interface to the R tool. You can access this at <https://rstudio.hpcc.ucr.edu>



Sign in to RStudio

Username:

Password:

You will automatically be signed out after 60 minutes of inactivity.

☐ Stay signed in when browser closes

[Sign In](#)

Login to Rstudio:

If you do not have access to HPCC you can still run a similar rstudio environment on rstudio.cloud. To save your code and support ease of moving code to/from rstudio.cloud environments it is best if you can setup a github environment for the project.

Practice UNIX steps

1. Generate a new directory
2. Navigate into the directory
3. How many lines are in the file `/bigdata/gen220/shared/simple/rice_random_exons.bed`
4. How many different (unique) chromosomes are there listed (The first column has the chromosome name)?